Stage 1: Circuit synthesis

Technical report · Group 2
Prepared for Zeyun DU (50025854). Project collaborator: Jincan Li.

> AI-assisted development. AI use has been confirmed for this project; this report records actual AI-assisted work and measurements. Students remain responsible for understanding and presenting the work.

Main result

The optimized program passes **25/25** published Stage 1 cases and meets **25/25** circuit-delay targets. The natural-order baseline also passes all cases but meets **12/25** targets. An independent checker confirms every output on all **3,395,584 input assignments**, and reproduces the official area and delay values.

Measure	Baseline	Optimized
Correct cases	25 / 25	25 / 25
Delay targets met	12 / 25	25 / 25
Sum of reported case areas	1,103,455	622,816
Median synthesis runtime	4.628 ms	603.945 ms

Total area falls by **43.6%**, and **23/25** cases have smaller circuits. The median per-case area ratio is **0.510**. Synthesis takes longer because the optimized program searches representations and mappings; all official runtime constraints are met.

Problem and contract

For n inputs and m outputs, construct a circuit equivalent to all 2^n truth-table rows using AND2, OR2 and NOT. Input x0 is the most significant bit. Signal 0 is constant zero; IDs 1…n are inputs. Gates have consecutive IDs and refer only to smaller IDs. The final line lists exactly m output signals.

Area is the sum of emitted gate costs; delay is the longest weighted path to an output. A delay-target violation affects quality but is not itself a correctness failure. Runtime and memory limits, when positive, are validity requirements. [Project handout, pp.1–3]

Method and correctness

Baseline: shared Shannon decisions

Recursively split the table in input order x0,…,x(n−1). Each decision has a low and high cofactor. Equal children eliminate the decision; identical triples (variable, low, high) share one node. Map a decision to:

f = (NOT x AND f0) OR (x AND f1).

Terminals and literal functions map directly. A structural gate table shares repeated operations, and a final reachability traversal removes unused gates and assigns consecutive output IDs.

Improved representation and mapping

- **Complement sharing:** one edge bit represents logical negation, reducing duplicated subfunctions.
- **Variable-order search:** compare natural, reverse, influence-based and seeded local/random orders. Influence samples propose orders; every candidate still uses the complete truth table.
- **Cost-aware logic:** compare sum-of-products, product-of-sums and De Morgan forms with the supplied gate costs and arrival times. Delay-oriented candidates can materialize both polarities instead of inserting an extra output inverter.
- **Two-level expansion:** replace two sequential decisions with up to four path-conditioned terms and a balanced OR tree. This trades some extra gates for fewer sequential multiplexer stages.
- **Algebraic structure:** verified cofactor implications simplify multiplexers; affine subfunctions become balanced XOR trees built from legal gates.
- **Associative balancing:** flatten single-fanout AND/OR cones and combine early-arriving signals first. Shared cones retain their boundaries.

Keep the completed baseline as a candidate. Rank complete, pruned circuits by delay excess, then area, then delay. Search uses at most 60% of the declared runtime allowance, capped at 1500 ms, with cooperative checks during BDD construction and mapping. A first valid candidate is built before optional search.

Why the transformations preserve the function

Shannon expansion selects exactly f0 for x=0 and f1 for x=1, so induction from the truth-table terminals proves the baseline. Complement normalization negates both children and the selected result. A four-way cover partitions assignments by two decisions, then ORs the selected cofactors. If f0 implies f1, f0 OR (x AND f1) selects the same value in both cases. Affine recognition checks that cofactors differ by a constant one. Associativity, idempotence, structural sharing and removal of unreachable gates preserve outputs.

The method is heuristic, not a proof of minimum area or delay. Fixed-order construction visits $O(m \cdot 2^n)$ table positions; BDD size can still be exponential. The measured limits and tests are therefore essential evidence.

Published results / 001–013

Every case below is **PASS** for both methods. Every optimized delay is at or below its target. Area and delay use the case-specific units; runtime is wall-clock program time measured by the official evaluator, excluding its correctness checks.

Case	Target	Base area	Opt area	Base delay	Opt delay	Base ms	Opt ms
001	77	135885	55838	71	76	18.79	1510.39
002	61	21835	28847	66	37	10.10	1505.18
003	64	28206	26059	66	55	11.18	1505.56
004	57	7713	3215	57	57	3.54	602.22
005	36	1594	590	36	34	1.47	61.78
006	60	14272	5218	56	60	3.50	602.97
007	89	231712	106573	89	85	140.70	1516.95
008	62	15924	6437	56	59	3.67	603.57
009	59	33212	13461	56	59	4.63	603.95
010	61	9394	4792	75	30	29.59	1502.48
011	40	620	436	47	33	2.55	61.64
012	40	4157	4442	46	25	2.45	602.67
013	70	11826	10700	67	56	7.86	1503.41

Reading the tradeoff

The selection rule first removes delay violations, then reduces area. It can choose a larger circuit when this is needed to meet the target. Individual case results, rather than only a total, show that tradeoff.

The area sum is a descriptive aggregate, not an official project score. Gate costs differ across cases, and no numerical grading formula or excellence threshold has been published in the supplied handout.

Measurement environment

Ubuntu 22.04.5 LTS, x86-64; GCC 11.4.0, `-O2 -std=c++17`. The course target is Ubuntu 22.04.3 with an AMD EPYC 7542 CPU. Measurements here are local, not claimed to be timings from that target server.

Published results / 014–025

All values come from the retained baseline and optimized evaluator CSVs. The independent checker recalculates area and delay from the actual emitted files.

Case	Target	Base area	Opt area	Base delay	Opt delay	Base ms	Opt ms
014	72	20808	11562	81	60	76.40	1504.29
015	81	38556	34395	76	81	24.30	1507.31
016	47	5008	2172	46	47	2.65	505.79
017	25	573	393	26	25	2.46	62.90
018	32	2933	1167	41	32	2.44	63.00
019	73	23692	4594	76	69	18.74	1503.04
020	35	824	634	36	35	2.64	61.73
021	72	69666	27966	72	60	20.93	1508.17
022	34	1286	1106	37	31	2.63	61.77
023	24	262	142	35	24	1.38	39.80
024	83	180822	165434	84	68	66.33	1532.37
025	76	242675	106643	71	76	27.43	1516.62

Timing and reproducibility

The optimized maximum reported runtime is **1532.367 ms**. All 25 runs satisfy their case runtime limits, including the 100 ms cases. A fixed random seed makes the search proposal sequence reproducible, but a wall-clock deadline can change how many proposals finish on another machine. Reproductions must retain their own emitted circuits and actual measurements.

The baseline is preserved as a separate executable and source file. Both programs are single-threaded C++17 and require no external synthesis package or network access. The source-only archive contains code, build instructions, checking scripts and retained numerical evidence; use the instructor's cases and evaluator for official reruns.

Review through experiments

Does a shorter search suffice?

Configuration	Pass	Target met	Area sum	Total runtime
Baseline	25/25	12/25	1,103,455	0.49 s
250 ms search cap	25/25	24/25	1,020,097	5.04 s
500 ms search cap	25/25	24/25	817,788	9.54 s
1500 ms cap (retained)	25/25	25/25	622,816	22.05 s

The 250 ms and 500 ms variants both miss case 024's delay target. The retained 1500 ms cap reaches all targets and produces smaller total area, at higher synthesis cost. This comparison supports the retained engineering choice; an unpublished runtime-weighted score could rank tradeoffs differently.

Independent correctness evidence

- **Published inputs:** all 3,395,584 assignments, every output. Independent results agree with official area and delay for both methods.
- **92 generated cases:** all Boolean functions on 1, 2 and 3 inputs under three gate-cost settings; 80 seeded random multi-output cases; structured cases on 4, 8 and 12 inputs. All pass full-table checking.
- **Checker self-tests:** six corrupt outputs are rejected, covering wrong behavior, nonconsecutive IDs, forward/self references, an illegal gate, output-count mismatch and invalid arity.
- **Sanitizers:** the generated regression suite also passes AddressSanitizer and UndefinedBehaviorSanitizer with leak detection enabled.
- **Largest input dimension:** a generated 20-input, 8-output case passes the official evaluator under a 10,000 ms runtime limit and 128 MiB memory cap. All 1,048,576 rows independently match; area 668, delay 68 ≤ 100.

Known limits

The tests do not prove optimality or predict hidden-case grades. BDD ordering can be costly for unstructured functions. The first construction and final output writing are not guaranteed to fit arbitrarily tiny supplied limits. The current synthesizer does not actively schedule against a positive RAM limit; the explicit memory-cap test adds evidence for one large structured instance, not a universal bound.

Reproduction and project gates

Commands

Run from the source root, with the teacher's evaluator and Stage 1 inputs in `official/`:

```
make
python3 tools/evaluate_release.py --output results/a-new-run
python3 tests/regression.py ./build/synth results/regression.json
```

The run directory must be new. Python verification uses NumPy; the synthesizer itself only needs the C++ standard library. `docs/METHOD.md` explains the transformations, and raw CSV/JSON logs retain the numerical evidence.

Completion review

Gate	Evidence / status
Stage 1 format, source and compile instructions	Standalone C++ sources and Makefile; README documents the interface
Method and all 25 case results	This report and retained official / independent outputs
Technical correctness and resource tests	Official, independent, generated, sanitizer and constrained stress checks
AI assistance	Confirmed for this project; students remain responsible for the work
Actual student teamwork	Canvas team known; contributions and division of work must reflect real activities
Stage 2 comparison	Additional 25 cases not yet released; both methods must be compared on all 50
Live Q&A	Date TBD; both students must attend
Hidden-case grading	Teacher evaluation, unavailable locally

Next-stage plan

Compare the actual adopted Stage 1 method with the Stage 2 improvement on all 50 public cases. The current optimized candidate is frozen in releases/stage1_candidate; the simple baseline remains an engineering ablation. Stage 2 should also address RAM-limit scheduling and an early valid fallback under very small runtime limits. Retain real teamwork records and prepare to explain the method, correctness, delay calculation and area/runtime tradeoff in the joint Q&A.

Sources: UFUG2106 Final Project, Canvas file 869384, pp.1–4; Course Information, Canvas file 785116. The syllabus assigns 15% of the course to the project; it does not provide an excellence cutoff.